

Supplementary IV

(CMA Secure CP-CLAS Schemes Implementation)

This project implements Zhang et al.'s, Chen et al.'s, and our CP-CLAS schemes. A total of 3 C source files are developed based on the PBC cryptography library. All schemes generate 40 user key pairs and 40 individual signatures, verify aggregate signatures composed of 2 to 40 individual signatures, and output the runtime of the **Sign** algorithm for 1 to 40 individual signatures, and the runtime of the **Aggregate-Verify** algorithm for 2 to 40 individual signatures (batch verification).

1 Prerequisites

Tested on **Ubuntu 24.04** (x86 64). Install required dependencies first:

1.1 Install GCC Compiler

```
sudo apt update
sudo apt install gcc make
```

1.2 Install PBC Library(Core Cryptography)

```
sudo apt install libpbc-dev
```

2 Source Code Files (Total: 3)

- **Zhang.C**: Implementation of Zhang et al.'s CP-CLAS scheme
- **Chen2.c**: Implementation of Chen et al.'s CP-CLAS scheme
- **CPCLAS2.c**: Implementation of our CP-CLAS scheme

3 Step-by-Step Compilation & Running

3.1 Step1: Compile the Source Code

Use gcc to compile the program, link **PBC** and **OpenSSL** libraries (critical for successful compilation):

```
gcc -o Zhang Zhang.c -lpbc -lgmp -lcrypto
```

```
gcc -o Chen2 Chen2.c -lpbc -lgmp -lcrypto
```

```
gcc -o CPCLAS2 CPCLAS2.c -lpbc -lgmp -lcrypto
```

- -lpbc: Links the PBC pairing cryptography library
- -lgmp: Links the GNU MP Bignum library
- -lcrypto: Links OpenSSL for hash functions

3.2 Step2: Run the Executable

```
./Zhang
```

```
./Chen2
```

```
./CPCLAS2
```

3.3 Expected Output

Each program performs the following operations, and outputs their runtimes:

1. System setup
2. Generate a pair of partial private key
3. Verify a pair of partial private key
4. Generate a pairs of user key
5. Generate 1-40 individual signatures
6. Aggregate 2-40 signatures and verify the aggregate signature

4 Screenshot Requirements

The runtime of 1-40 Sign algorithms and an Aggregate-Verify algorithm for 2-40 individual signatures.

```

spiderman@spiderman-VMware-Virtual-Platform:~/Downloads/pbc-0.5.14/include$ gcc -o Zhang
Zhang Zhang.c -lpbc -lgmp -lcrypto
spiderman@spiderman-VMware-Virtual-Platform:~/Downloads/pbc-0.5.14/include$ ./Zhang
Execution time of Setup algorithm: 0.004440 seconds
Execution time of an extraction algorithm from TA's side: 0.000083 seconds
Execution time of an extraction algorithm from Vehicle's side: 0.000033 seconds
Execution time of a key generation algorithm from Vehicle's side: 0.004687 seconds
execution times of 0-40 signing algorithms is:
0.090869 seconds
0.186945 seconds
0.280330 seconds
0.372960 seconds
0.466044 seconds
0.558793 seconds

```

```

0.184088 seconds
0.191290 seconds
spiderman@spiderman-VMware-Virtual-Platform:~/Downloads/pbc-0.5.14/include$ gcc -o Chen2
Chen2 Chen2.c -lpbc -lgmp -lcrypto
spiderman@spiderman-VMware-Virtual-Platform:~/Downloads/pbc-0.5.14/include$ ./Chen2
Execution time of Setup algorithm: 0.004479 seconds
Execution time of an extraction algorithm from TA's side: 0.000073 seconds
Execution time of an extraction algorithm from Vehicle's side: 0.000029 seconds
Execution time of a key generation algorithm from Vehicle's side: 0.009151 seconds
execution times of 1-40 signing algorithms:
0.064009 seconds
0.125150 seconds
0.199790 seconds
0.267274 seconds
0.328935 seconds

```

```

0.157111 seconds
0.163765 seconds
spiderman@spiderman-VMware-Virtual-Platform:~/Downloads/pbc-0.5.14/include$ gcc -o CP
CLAS2 CPCLAS2.c -lpbc -lgmp -lcrypto
spiderman@spiderman-VMware-Virtual-Platform:~/Downloads/pbc-0.5.14/include$ ./CPCLAS2
Execution time of Setup algorithm: 0.004515 seconds
Execution time of an extraction algorithm from TA's side: 0.000154 seconds
Execution time of an extraction algorithm from Vehicle's side: 0.000080 seconds
Execution time of a key generation algorithm from Vehicle's side: 0.009020 seconds
execution times of 1-40 signing algorithms:
0.065018 seconds
0.135582 seconds
0.201736 seconds
0.268631 seconds
0.335284 seconds

```

```

3.693152 seconds
3.791356 seconds
execution times of an agg-verificaiton algorithm of 2-40 signatures:
0.003314 seconds
0.006844 seconds
0.010374 seconds
0.013353 seconds
0.016504 seconds
0.019584 seconds
0.023103 seconds

```

5 Comparisons

5.1 Test Environment

- **Operating System:** Ubuntu 24.04
- **Processor:** a single-core at 1.80-GHz
- **Random Access Memory:** 2GB
- **Elliptic Curve:** Type-A
- **Libraries:** PBC, GMP

5.2 Results

The following tables show **performance comparison** among Zhang et al.'s, Chen et al.'s, and our CP-CLAS schemes (s).

Table 1. Signing Cost (s)

Schemes	Zhang	Chen	CP-CLAS
5 messages	0.466044	0.328935	0.335284
10 messages	0.929243	0.674431	0.676966
15 messages	1.420609	1.011575	1.023949
20 messages	1.887719	1.352602	1.374344
25 messages	2.362466	1.687867	1.704729
30 messages	2.832979	2.021035	2.046205
35 messages	3.305688	2.352214	2.384094
40 messages	3.791356	2.698041	2.729300

Table 2. Verification Cost (s)

Schemes	Zhang	Chen	CP-CLAS
5 messages	0.013353	0.008366	0.008946
10 messages	0.030420	0.021603	0.022934
15 messages	0.049951	0.037830	0.039800
20 messages	0.072166	0.056788	0.059344
25 messages	0.097658	0.078628	0.083118
30 messages	0.125627	0.103842	0.118203
35 messages	0.156892	0.132895	0.158098
40 messages	0.191290	0.163765	0.190260

5.3 Observation:

- **Table1:** The difference in signing cost between Chen et al.'s scheme and our CP-CLAS scheme is negligible.
- **Table1:** Our scheme has a lower signing cost than Zhang et al.'s scheme.
- **Table2:** As the number of vehicles increases, the gap in verification cost among the three schemes narrows.